Universidad Zaragoza

INTELIGENCIA ARTIFICIAL (30223) – Curso 2026-27

Grado en Ingeniería Informática

Dpto. Informática e Ingeniería de Sistemas

Práctica 1 : Resolución de problemas y búsqueda. Búsqueda no informada

1. Objetivo de la práctica

Familiarizarse con el código en **Python** para resolver problemas de búsqueda. El repositorio base se encuentra en https://github.com/aimacode/aima-python. Debes clonar o descargar este repositorio para disponer de las herramientas estándar de AIMA (`search.py`, `utils.py`, etc.).

En esta práctica se pretende:

- Familiarizarte con los algoritmos de búsqueda. Puedes abrir el notebook `search.ipynb`, estudiar el código y ejecutar las celdas.
- Instrumentar el código: Modificar las funciones de búsqueda no informada de `search.py` para que, además de devolver el nodo solución, devuelvan una tupla con las métricas necesarias para evaluar el rendimiento del algoritmo.
- Modelar problemas: Utilizar y definir diferentes algoritmos de búsqueda ciega para resolver problemas como los Caníbales y Misioneros, el 8-puzzle y el 15-puzzle.
- Evaluar el rendimiento: Analizar comparativamente la eficiencia temporal y espacial de cada algoritmo basándose en las métricas obtenidas y extraer conclusiones. Se pretende (i) utilizar distintos algoritmos de búsqueda ciega, (ii) ser capaz de definir un problema y resolverlo con los algoritmos de búsqueda, y (iii) evaluar la aplicación de los algoritmos a los problemas y extraer algunas conclusiones. En esta práctica trataremos de aplicar diferentes algoritmos de búsqueda no informada para el problema del **8-puzzle**, obtendremos métricas de complejidad temporal y espacial, y definiremos el problema para **los caníbales y los misioneros**.

8-puzzle

El 8-puzzle es un juego de mesa para un único jugador que consiste en 8 piezas cuadradas numeradas del 1 al 8 y un espacio vacío en un tablero de 3 x 3. El objetivo del juego es, partiendo de un estado representado por una disposición determinada de las piezas (estado inicial I) realizar movimientos permitidos para alcanzar una configuración deseada (meta O) en el menor número de movimientos posible.

La descripción de un estado especifica la localización de cada una de las 8 fichas y el espacio en blanco, en cada uno de los nueve cuadrados.

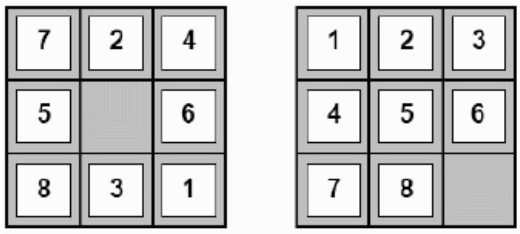


Figura 1: Ejemplos de estado inicial y estado final del 8-puzzle

Un **estado inicial** es cualquier configuración de las fichas de partida y un **estado meta**, una configuración determinada que hay que alcanzar (ver Figura 1). Para representar el problema podemos utilizar un array con los números {0,1,2,3,4,5,6,7,8} donde el cero simboliza la casilla vacía.

2. Tareas algoritmos búsqueda no informada

1. En `aima-python`, la clase `EightPuzzle` ya viene predefinida en `search.py`. El estado se representa como una **tupla de 9 elementos** donde el `0` representa la casilla vacía

2. Debes probar los algoritmos con tres estados iniciales cuya solución se encuentra a 3, 9 y 30 pasos respectivamente:
   threeSteps = (1, 2, 5, 3, 4, 0, 6, 7, 8)

   nineSteps = (1, 4, 2, 7, 5, 8, 3, 0, 6)

   thirtySteps =(0, 8, 7, 6, 5, 4, 3, 2, 1)

   Ojo, en l línea 521 de search.py debes modificar el estado final a (0,1,2,3,4,5,6,7,8), pon el cero al principio.

3. Prueba los algoritmos de búsqueda no informada vistas (anchura, profundidad en árbol y en grafo, búsqueda en profundidad recursiva con límite (DLS), la búsqueda iterativa en profundidad IDLS y coste uniforme) con estos estados iniciales y visualiza una tabla como la mostrada con las métricas obtenidas: Profundidad (coste de la solución), Expand (Nodos expandidos), Gen (Nodos Generados), MaxQS (Tamaño máximo frontera), y tiempo de ejecución. Habrá algoritmos que no sean capaces de resolver en tiempo razonable o no tengan suficiente memoria. En esos casos pon un comentario (1), o (2) para hacer referencia a esos casos. Comenta los resultados obtenidos. Crea un script Python (P1_table.py) con las métricas pedidas. **En la memoria, debes comentar cada algoritmo-problema, y comparar los resultados con la complejidad temporal y espacial "teórica".** Reflexiona sobre los resultados. Ejemplo de ejecución:

Problema	Profundidad	Expand	Q.size	MaxQS	Tiempo	Generados
BFS-G-3	3	4	5	5	0.0000	11
BFS-T-3	3	16	31	32	0.0000	48
DFS-G-3	59123	120491	39830	42913	434.6064	160322
DFS-T-3	---	---	---	---	---	(1)
DLS-3-3	3	4	0	0	0.0000	11
DLS-9-3	9	10	0	0	0.0000	26
IDS-3	3	4	0	0	0.0001	11
UCS-3	3	9	9	10	0.0001	19
BFS-G-9	9	287	198	199	0.0046	487
BFS-T-9	9	16877	29690	29691	0.0577	46568
DFS-G-9	44665	141452	32012	42967	519.7320	173465
DFS-T-9	---	---	---	---	---	(1)
DLS-3-9	---	---	---	---	---	(3)
DLS-9-9	9	5474	0	0	0.0252	15924
IDS-9	9	5474	0	0	0.0406	15924
UCS-9	9	291	201	202	0.0236	493
BFS-G-30	30	181057	365	24048	289.7359	181424
BFS-T-30	---	---	---	---	---	(2)
DFS-G-30	62856	80569	41533	41534	286.8132	122103
DFS-T-30	---	---	---	---	---	(1)
DLS-3-30	---	---	---	---	---	(3)
DLS-9-30	---	---	---	---	---	(3)
IDS-30	---	---	---	---	---	(3)
UCS-30	30	181217	220	24107	1198.7107	181438

NOTAS:
Para visualizas las métricas de la búsqueda puedes modificar cada uno de los algoritmos para que devuelva una tupla

añadiendo las métricas pedidas. Las métricas pueden ser devueltas como valores en orden o como un diccionario de Python que permite asociar valores a claves. Ej.: Metric = {“Depth”: 3, “Expand”:5}.

Misioneros y Caníbales

En el problema de los misioneros y los caníbales se plantea el siguiente problema: En la orilla de un río hay 3 misioneros y 3 caníbales y todos ellos pretenden cruzar al otro lado. La barca que se utiliza para cruzarlo solo tiene capacidad para dos personas, con lo que alguien ha de estar volviendo siempre a la orilla inicial mientras quede gente sin cruzar. Además, si en alguna ocasión y en cualquiera de las orillas se encuentran un número mayor de caníbales que de misioneros, los primeros se comerán a los segundos.

3. Tareas de definición de problemas

Debes implementar y completar el script `P1_canibales.py` creando la clase `Canibals` heredada de `Problem`. Estudia la clase `Problem` para entender que funciones y valores debes implementar en la clase Canibals.

1. **Estado:** Represéntalo como una `tupla (C_izq, M_izq, Bote, C_der, M_der)`, donde Bote es `0` si está en la orilla izquierda y `1` si está en la derecha.
2. **Acciones:** `['1C', '2C', '1C1M', '1M', '2M']`.
3. **Impresión de la solución:** Define una función `print_state(state)` y recorre los nodos de la solución recuperados mediante `X.path()` para imprimir el camino encontrado y las acciones aplicadas.

Ejemplo de ejecución:

```
None  |RIBERA-IZQ M M M C C C BOTE --RIO--                  RIBERA-DCH
  2C  |RIBERA-IZQ M M M     C      --RIO-- BOTE         C C RIBERA-DCH
  1C  |RIBERA-IZQ M M M   C C BOTE --RIO--                C RIBERA-DCH
  2C  |RIBERA-IZQ M M M            --RIO-- BOTE       C C C RIBERA-DCH
  1C  |RIBERA-IZQ M M M     C BOTE --RIO--               C C RIBERA-DCH
  2M  |RIBERA-IZQ     M     C      --RIO-- BOTE   M M   C C RIBERA-DCH
1C1M  |RIBERA-IZQ   M M   C C BOTE --RIO--           M     C RIBERA-DCH
  2M  |RIBERA-IZQ         C C      --RIO-- BOTE M M M     C RIBERA-DCH
  1C  |RIBERA-IZQ       C C C BOTE --RIO--      M M M       RIBERA-DCH
  2C  |RIBERA-IZQ           C      --RIO-- BOTE M M M   C C RIBERA-DCH
  1C  |RIBERA-IZQ         C C BOTE --RIO--      M M M     C RIBERA-DCH
  2C  |RIBERA-IZQ                  --RIO-- BOTE M M M C C C RIBERA-DCH
```

Se deben entregar como primera práctica un fichero zip que tenga por nombre `P1_nip.zip`, reemplazando “nip” por el tuyo. Se debe incluir las clases escritas o modificadas y una breve memoria escrita con las tareas realizadas comentando tus resultados obtenidos al utilizar distintos algoritmos. Por ejemplo, puedes copiar el fichero search.py y renombrarlo `searchEina.py` para incluir en este último las métricas pedidas. Entrega searchEina.py junto a los ficheros `P1_table.py` (genera la traza tarea 2) y `P1_canibales.py` (muestra traza tarea 3). Las entregas se deben poder ejecutar en si mismas para que sean validas.